

Interpreter - Penetration Tester HTB

1. Executive Summary

During the security assessment of the **Interpreter** infrastructure, a chain of vulnerabilities was identified within a web application's API and an internal management script that allowed for unauthorized system access and full administrative compromise. The attack path involved exploiting an insecure XML deserialization flaw (CVE-2023-43208) in the NextGen Connect Client API, exfiltrating database credentials to pivot to a local user, and finally exploiting a Python f-string injection vulnerability in a privileged script to gain root access.

2. Technical Findings and Exploitation

2.1 Remote Code Execution (RCE): XML Deserialization (CVE-2023-43208)

Severity: Critical

- **Observation:** The application was running NextGen Connect Client API version 4.4.0, which is vulnerable to unauthenticated remote code execution via insecure XML deserialization.
- **Impact:** This allowed for an initial foothold on the Linux host under the context of the web service user.
- **Exploitation:** By sending a specially crafted XML payload to the API endpoint, the vulnerability was triggered, resulting in a reverse shell connection back to the attacker's infrastructure.
- **Remediation:** Upgrade NextGen Connect to a version where XML deserialization is handled securely and ensure that the API is not exposed to the public internet without proper authentication.

2.2 Information Disclosure: Plaintext Configuration Secrets

Severity: High

- **Observation:** A search of the application's configuration directories revealed a file named `mirth.properties` within the `/conf` folder.
- **Impact:** This file contained sensitive database credentials and host information in plaintext.
- **Exploitation:** The following credentials were exfiltrated:
 - **DB Name:** `mc_bdd_prod` | **User:** `mirthdb` | **Password:** `MirthPass123!`
- **Remediation:** Encrypt sensitive configuration values and restrict file system permissions (e.g., `chmod 600`) to ensure only the necessary service accounts can read them.

2.3 Broken Authentication: Weak Password Hashing/Dictionary Attack

Severity: High

- **Observation:** Although the system utilized PBKDF2-HMAC-SHA256 with 600,000 iterations, the application was susceptible to a dictionary attack due to a weak password policy.
 - **Impact:** Allowed for lateral movement from the service account to a local user account (`sedric`).
 - **Exploitation:** After extracting password hashes from the MariaDB instance, the `rockyou.txt` wordlist was used to successfully crack the password for the user "sedric" (`snowflake1`).
 - **Remediation:** Implement a robust password complexity policy and encourage the use of unique, long, and complex passphrases.
-

3. Privilege Escalation

3.1 Local Privilege Escalation: Python f-string Injection

Severity: Critical

- **Observation:** An internal service listening on port 54321 was linked to a Python script located at `/usr/local/bin/notif.py` . The script used an f-string template to process notifications.
- **Impact:** Because f-strings can execute code within curly braces `{}` and the script ran with elevated privileges, this provided a direct path to root.
- **Exploitation:** A multi-stage payload was designed:
 1. **Payload:** A command to copy `/bin/bash` to `/tmp` and set SUID permissions (`chmod +4777`).
 2. **Encoding:** The payload was Base64 encoded to bypass XML character filters.
 3. **Execution:** The encoded command was injected into the notification template, which the script evaluated and executed as root.
- **Remediation:** Avoid using `eval()` or dynamic f-string evaluation with user-supplied input. Use standard logging libraries or templating engines that treat input strictly as data.

4. Remediation Summary

Finding	Severity	Recommendation
NextGen Connect RCE	Critical	Update to a patched version of NextGen Connect; restrict API access.
Plaintext Credentials	High	Use a secrets management solution and restrict file permissions.
Weak Password Policy	High	Enforce stronger password complexity requirements.
Python Code Injection	Critical	Refactor <code>notif.py</code> to remove dynamic code evaluation logic.